



The Ghost in the Machine: Technical Debt, Vibe Coding, and the Lessons We Keep Refusing to Learn

By Billy Miller | Delivery Leader | March 2026

There is a particular kind of hubris that lives in the gap between "it works" and "it's right." I've seen it in every generation of developer tooling and right now, in 2026, we are living through its most seductive and most dangerous iteration yet.

Vibe coding is real. The productivity gains are real. And the technical debt it generates silently, invisibly, at machine speed is more real than most teams are ready to admit.

Let me take you back to where we've been before. Because this isn't the first time we built a rocket ship out of scaffolding and called it architecture.

RAD, .NET Skeletons, and the Original Sin of Generated Code

Cast your mind back to Visual Studio 2003. You right-click a project, hit **Add New Item**, and a wizard generates a Windows Forms application complete with `InitializeComponent()`, a designer-generated `Form1.cs`, event handler stubs, and a `Program.cs` with a `Main()` method that nobody ever read.

Or better yet: you scaffold a Web Forms project and Visual Studio hands you a `Global.asax`, a `Web.config` with 200 lines of boilerplate, code-behind files tightly coupled to your UI, and a `DataSet` hooked directly to a SQL Server stored procedure that someone's manager named `sp_GetStuff`.

This was Rapid Application Development. It was glorious and it was a disaster.

The promise was real: compress months of plumbing into minutes. Let developers focus on business logic instead of infrastructure ceremony. Ship faster.

The reality, three years into a production system, looked like this:

```
// Visual Studio 2003 - Auto-generated Windows Forms Designer code
// Nobody touched this. Nobody understood this. It ran in production for 11
// years.

private void InitializeComponent()
{
    this.btnSubmit = new System.Windows.Forms.Button();
    this.txtName = new System.Windows.Forms.TextBox();
    this.lblName = new System.Windows.Forms.Label();
    this.dataGrid1 = new System.Windows.Forms.DataGrid();
    this.dataSet11 = new System.Data.DataSet();
    this.dataTable1 = new System.Data.DataTable();
    this.dataColumn1 = new System.Data.DataColumn();
    this.dataColumn2 = new System.Data.DataColumn();
    // ... 340 more lines of this
    ((System.ComponentModel.ISupportInitialize)(this.dataSet11)).BeginInit();

    ((System.ComponentModel.ISupportInitialize)(this.dataTable1)).BeginInit();
    this.SuspendLayout();
    // Event bindings nobody wrote and nobody can trace
    this.btnSubmit.Click += new System.EventHandler(this.btnSubmit_Click);
    // ...
}
```

You didn't write it. You didn't fully understand it. But it was *your* codebase now and when it broke at 2am, you owned every line.

The technical debt wasn't just in the generated code. It was structural. The scaffolding made architectural decisions *for* you tightly coupled UI to data, no separation of concerns, no

testability, no meaningful abstraction boundaries. It baked assumptions into your application that you'd spend years trying to refactor out.

Sound familiar?

Enter Vibe Coding: RAD at the Speed of Language

Vibe coding the practice of describing what you want in natural language and letting an AI model generate the implementation is the most powerful developer productivity tool since the compiler. I use it. My team uses it. It genuinely compresses hours into minutes for the right class of problems.

But here is what nobody is saying loudly enough:

A language model generates code the way Visual Studio generated scaffolding optimized for plausibility, not for your system.

The model doesn't know your architecture. It doesn't know your team's conventions. It doesn't know that you already have a `UserContext` service that handles the thing it just reimplemented inline. It doesn't know that you're about to scale to 10 million users and the pattern it chose will bring your database to its knees.

It knows what code that *looks like a solution to your problem* tends to look like. And it generates that. Confidently. Completely. With zero understanding of the system it's landing in.

Let me show you what this looks like in practice.

The Debt, Side by Side

Example 1: Fetching Data

The Vibe Coding Output (*what you get when you say "write me a function to get user orders"*)

```
import requests
import json

def get_user_orders(user_id):
    response =
requests.get(f"https://api.myapp.com/orders?user_id={user_id}")
    if response.status_code == 200:
        data = json.loads(response.text)
        orders = []
        for item in data['orders']:
```

```

        order = {
            'id': item['id'],
            'total': item['total_amount'],
            'status': item['order_status'],
            'date': item['created_at']
        }
        orders.append(order)
    return orders
else:
    return []

```

Looks reasonable. Ships in 90 seconds. Now count the debt:

- Hardcoded URL environment-blind, untestable, will break on every deploy
- `json.loads(response.text)` ignores `response.json()`, adds fragility
- Silent failure on non-200 caller has no idea if the request failed or returned empty
- No timeout this will hang indefinitely in production
- No retry logic one flaky network moment = silent data loss
- Manual dict construction instead of a typed model no validation, no IDE support
- No authentication header this almost certainly won't work against a real API

The Architecture-Aware Version *(what a senior engineer actually ships)*

```

from typing import List
from dataclasses import dataclass
from datetime import datetime
import httpx
from tenacity import retry, stop_after_attempt, wait_exponential

@dataclass
class Order:
    id: str
    total: float
    status: str
    created_at: datetime

class OrdersClient:
    def __init__(self, base_url: str, api_key: str, timeout: float = 5.0):
        self._client = httpx.Client(
            base_url=base_url,
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=timeout
        )

        @retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1,
min=1, max=4))
        def get_user_orders(self, user_id: str) -> List[Order]:
            response = self._client.get("/orders", params={"user_id": user_id})
            response.raise_for_status()
            return [
                Order(
                    id=item["id"],
                    total=item["total_amount"],
                    status=item["order_status"],

```

```

        created_at=datetime.fromisoformat(item["created_at"])
    )
    for item in response.json()["orders"]
]

```

The delta between these two functions is not style. It is the difference between something that *demoes* and something that *operates*.

Example 2: The Scaffolding Problem Hasn't Gone Away

The old Visual Studio skeleton for an ASP.NET Web API controller looked like this:

```

// VS 2013 Auto-scaffolded API Controller – every project got this
public class ValuesController : ApiController
{
    // GET api/values
    public IEnumerable<string> Get()
    {
        return new string[] { "value1", "value2" };
    }

    // GET api/values/5
    public string Get(int id)
    {
        return "value";
    }

    // POST api/values
    public void Post([FromBody]string value) { }

    // PUT api/values/5
    public void Put(int id, [FromBody]string value) { }

    // DELETE api/values/5
    public void Delete(int id) { }
}

```

No dependency injection. No validation. No error handling. No logging. No authorization. Synchronous. Returning `void` on mutations. This was the *starting point* for thousands of production APIs.

The AI-generated equivalent in 2026 isn't structurally different:

```

# Vibe coded FastAPI endpoint – "build me a REST API for orders"
from fastapi import FastAPI

app = FastAPI()

orders_db = {} # In-memory storage

@app.get("/orders/{order_id}")

```

```
def get_order(order_id: str):
    return orders_db.get(order_id, {})

@app.post("/orders")
def create_order(order: dict):
    order_id = str(len(orders_db) + 1)
    orders_db[order_id] = order
    return {"id": order_id}

@app.delete("/orders/{order_id}")
def delete_order(order_id: str):
    del orders_db[order_id]
```

In-memory dict masquerading as a database. No auth. No validation. `del` on a key that may not exist. No status codes on responses. No pagination. The app will lose all data on restart and throw a `KeyError` on the first delete of a non-existent order.

The model did exactly what you asked. The model had no idea what you needed.

Why This Happens: The Model Doesn't Know What It Doesn't Know About You

Language models are trained on the average of human-written code. They generate the most *statistically plausible* solution which means they generate what a competent junior developer would write when solving the problem in isolation.

Your system is not isolated. It has:

- Existing abstractions the new code should use
- Architectural patterns it should conform to
- Non-functional requirements (latency, scale, resilience) it must meet
- Regulatory or compliance constraints
- A team that has to maintain it for the next five years

The model knows none of this unless you tell it. And most developers especially under time pressure don't tell it enough.

Five Prompt Patterns That Cut Debt at the Source

The good news: this is solvable. Not perfectly, not automatically, but meaningfully. Here is how senior engineers I respect are thinking about their prompts.

1. Lead with Constraints, Not Just the Task

Debt-generating prompt:

"Write a function to authenticate a user."

Debt-reducing prompt:

"Write a function to authenticate a user. Use our existing `UserRepository` interface for data access. Return a `Result<AuthToken, AuthError>` type do not throw exceptions. Assume the caller handles retry logic. Target Python 3.11, `async/await`, and include type hints throughout. The function will be called from a FastAPI route handler under 200ms SLA."

The second prompt constrains the solution space before the model writes a single character. You're not describing the task you're describing the *system*.

2. Specify What You Already Have

Debt-generating prompt:

"Write me a logging utility."

Debt-reducing prompt:

"We use `structlog` for structured logging throughout the codebase, configured in `app/logging.py`. Write a context manager that adds a `correlation_id` field to all log statements within its scope. Match the existing pattern in `app/middleware/tracing.py`. Do not introduce any new logging libraries."

Every new abstraction the model invents is debt. Tell it what already exists.

3. Ask for the Test First

Before asking for the implementation, ask for the test:

"Write the unit tests for a function that validates an email address per RFC 5321. Include edge cases: empty string, no `@` symbol, multiple `@` symbols, international domains, and addresses over 254 characters. Use `pytest`. I'll implement the function to pass these tests."

Now you have a specification. Now you have something that constrains what the implementation can be. The model's generated code has to satisfy concrete, explicit requirements rather than its own interpretation of your vague ask.

4. Request Explicit Non-Functional Requirements

"The function will be called 50,000 times per second at peak load. Optimize for memory allocation over readability. Do not create intermediate collections. Show me the time complexity of your approach."

Most vibe-coded functions are $O(n^2)$ because the model chose the readable approach, not the scalable one. You have to ask.

5. Force Architectural Alignment with a System Prompt

If you're working in a tool that accepts a system prompt or context window, front-load your architectural decisions:

You are writing code for a Python 3.11 FastAPI service.

Architecture:

- Repository pattern for all data access
- Services are stateless and injectable via FastAPI's `Depends()`
- All external I/O is `async`
- Errors are returned as typed `Result` objects, never raised as exceptions
- Pydantic v2 for all data models
- No global state
- Every public function has type hints and a docstring

When I ask you to implement something, follow these patterns exactly.
If a request would require violating these patterns, tell me before writing code.

This doesn't eliminate debt. But it gives the model the architectural rails your system actually runs on.

The Refactoring Tax

Here is the number most organizations aren't tracking: the cost of cleaning up vibe-coded debt is not linear with the debt incurred. It compounds.

Vibe-coded function A calls vibe-coded service B which was wired up by vibe-coded configuration C. None of them follow your architecture. All of them work right up until they don't. The refactor requires understanding three systems you didn't write, can't fully explain, and have no tests for.

This is the same tax we paid with RAD. The same tax we paid with Visual Studio scaffolding. The same tax we'll pay with whatever the next productivity acceleration looks like.

The lesson has never been "don't use the tool." The lesson has always been: **the tool generates starting points, not endpoints. A senior engineer's job is to know the difference.**

The Bottom Line

Vibe coding is the most significant shift in developer productivity since the IDE. Use it. Embrace it. Build faster with it.

But treat every line it generates the way a senior code reviewer would with the understanding that the model optimized for solving your stated problem, not for living inside your system. The gap between those two things is where technical debt is born.

The old RAD tools gave us productivity and gave us mess. We cleaned up the mess over a decade of refactoring, design patterns, and hard-won architectural discipline. We don't have a decade this time.

The engineers who will define the next ten years of software aren't the ones who can prompt the fastest. They're the ones who know when to stop accepting the first answer and how to ask a better question.

Thoughts? I'd love to hear how your team is managing AI-generated technical debt — drop a comment below or reach out directly.

*#SoftwareArchitecture #TechnicalDebt #VibeCoding #AI #GenerativeAI #SoftwareEngineering
#CloudArchitecture #DeveloperProductivity*